

Hypothétique LAB 5-6-7

LAB 5 - Migration vers une Architecture Microservices

1. Découpage logique du système

Identification de **4 services principaux** dans le système :

- **Gestion des produits** (CRUD produits, catégories)
- **Gestion des ventes** (transactions, facturation)
- **Gestion du stock** (inventaire partagé)
- **Reporting** (statistiques, rapports)

Séparation logique dans le projet :

- Chaque service placé dans un module ou package dédié
- Chaque service empaqueté dans son propre conteneur Docker pour un déploiement indépendant

Ajout de 3 nouvelles API :

- **Création de comptes clients** (POST /clients)
- **Gestion du panier d'achat** (POST /panier, GET /panier)
- **Validation de commande (checkout)** (POST /commande/valider)

Intégration de la coexistence entre services :

Le service de stock est commun au magasin physique et à la boutique en ligne.

2. Mise en place de l'API Gateway

- Choix d'une API Gateway open-source
- Configuration d'un point d'entrée unique redirigeant les requêtes vers les microservices correspondants

Implémentation de deux fonctionnalités :

- **Routage dynamique** → en fonction des chemins (/produits, /panier, etc.)
- **Ajout d'en-têtes d'authentification** ou gestion de clés API

Mise en place d'un logging centralisé pour tracer toutes les requêtes.

3. Load balancing via l'API Gateway

- Mise en place d'un round-robin entre deux instances du service "gestion du panier"
- Documentation de la configuration de répartition de charge

Test de distribution de charge avec k6 :

- Simulation de plusieurs utilisateurs simultanés

- Observation du partage des requêtes entre les instances

4. Sécurité et gestion des accès

Configuration des règles CORS dans la Gateway pour contrôler l'accès depuis des domaines autorisés.

5. Documentation et tests

- Mise à jour du fichier Swagger/OpenAPI pour inclure les nouvelles API
- Adaptation des requêtes Postman pour tester les services via la Gateway
- Vérification que toutes les routes fonctionnent correctement à travers l'API Gateway

6. Observabilité et comparaison

Réutilisation de Prometheus et Grafana pour suivre :

- **Latence**
- **Disponibilité**
- **Taux d'erreurs**
- **Traçabilité des appels** (via logs centralisés)

Réalisation de deux tests de charge :

- Appels directs aux API (ancienne architecture monolithique)
- Appels via API Gateway (nouvelle architecture microservices)

Analyse comparative :

- **Temps de réponse** : légère hausse via la Gateway (overhead), mais meilleure répartition de charge
- **Taux d'erreurs** réduit en cas de forte charge grâce au load balancing
- **Visibilité accrue** via Grafana (monitoring centralisé)

Présentation des résultats dans un dashboard Grafana + captures d'écran intégrées au rapport.

LAB 6 - Saga Orchestrée

1. Définition du scénario métier

Choix d'un scénario concret : **Création de commande client.**

Étapes principales :

1. **Vérification du stock** → s'assurer que le produit est disponible
2. **Réservation du stock** → bloquer les quantités demandées
3. **Paiement** → demander au service de paiement de débiter le client
4. **Confirmation ou annulation** de la commande selon le résultat

Services impliqués :

Chaque étape est gérée par un microservice distinct :

- **StockService** (vérification & réservation)
- **PaieementService** (paiement)
- **CommandeService** (confirmation / annulation)

Stock partagé entre boutique en ligne et magasin physique.

2. Mise en place de la Saga orchestrée

Création d'un service orchestrateur dédié (ex : **OrchestrateurCommande**).

Fonctionnement :

- Reçoit un événement initial **CommandeCréée**
- Appelle chaque microservice en séquence contrôlée (synchrone)
- Sur succès complet → état final = **CommandeConfirmée**
- Sur échec (ex : paiement refusé) → déclenche des actions de compensation (rollback), par exemple libérer le stock

L'orchestrateur est responsable de la logique métier transversale.

3. Gestion des événements et de l'état

- Chaque microservice publie un événement (succès ou échec) après traitement
- L'orchestrateur maintient un état courant de la saga via une **machine d'état** :

États possibles :

EN_ATTENTE → **STOCK_RÉSERVÉ** → **PAYÉ** → **ANNULÉ** / **CONFIRMÉ**

- Machine d'état persistée en base (SQL ou NoSQL) pour suivi et reprise après incident
- Logs explicites pour chaque transition d'état

4. Simulation de cas d'échec

Mise en place de défaillances contrôlées :

- **Stock insuffisant** → étape de réservation échoue
- **Paiement refusé** → déclenche rollback

Observation des effets sur :

- État de la saga (passage en **ANNULÉ**)
- Logs métier retraçant toutes les décisions

Actions de compensation :

- Annuler la commande
- Libérer le stock réservé

5. Observabilité et traçabilité

Ajout de métriques Prometheus :

- Nombre total de sagas exécutées
- Durée moyenne d'une saga
- Taux d'échec par étape
- Étapes atteintes avant échec

Visualisation dans Grafana :

- Graphique temps moyen de traitement par étape
- Graphique taux d'échec / succès

Ajout de logs structurés pour tracer :

- Chaque événement métier
 - Les décisions prises par l'orchestrateur
 - Les actions de compensation exécutées
-

LAB 7 - Event Sourcing et CQRS

Partie 1 – Event Sourcing et CQRS

1. Définition du scénario métier

Choix d'un scénario concret : **Panier e-commerce**.

Étapes métier :

- Ajout d'articles au panier
- Paiement
- Expiration automatique si paiement non effectué dans un délai

Événements clés définis :

- **ArticleAjoute**
- **PaieementEffectue**
- **PanierExpire**

2. Implémentation des producteurs d'événements

- Création d'un service producteur (ex : **PanierService**) qui publie des événements métier
- Utilisation d'un système de messagerie **Kafka** (ou RabbitMQ / Redis Streams)

Chaque événement est :

- Sérialisé en JSON
- Horodaté
- Associé à un ID unique

Organisation des topics :

- panier.events
- paiement.events

3. Implémentation des abonnés aux événements

Création d'au moins deux consommateurs :

- **NotificationService** : envoie un email ou message Slack au client
- **AuditService** : enregistre tous les événements dans un log structuré JSON

Consommateurs conçus pour être :

- **Idempotents** (même événement traité deux fois = pas d'effet secondaire)
- **Robustes** (gestion d'erreurs, reconnexion automatique)

Chaque événement traité est journalisé avec contexte.

4. Mise en œuvre d'un Event Store

- Chaque événement est enregistré dans une base dédiée (ex : MongoDB ou PostgreSQL)
- Implémentation d'un mécanisme de **replay** :
 - Permet de reconstruire l'état métier d'un panier ou d'une commande à partir de l'historique complet

Ajout d'un endpoint REST pour afficher l'état courant reconstruit.

5. Implémentation de CQRS avec Event Broker**Séparation des responsabilités :**

- **Command** : exécution d'actions métier (ajout d'article, paiement...)
- **Query** : lecture d'un modèle optimisé (read model)

Utilisation des événements comme source de vérité pour mettre à jour les read models.

Les requêtes ne passent pas par la logique métier, mais directement par :

- Projections
- Caches
- Indexes optimisés

6. Ajout d'observabilité**Instrumentation avec Prometheus :**

- Nombre d'événements publiés / consommés
- Latence entre émission et consommation

Création d'un dashboard Grafana :

- Activité des topics Kafka
- Volume de messages
- Latence moyenne

Logs structurés pour suivre le cycle de vie complet de chaque événement.

Partie 2 – Saga Chorégraphiée**1. Analyse du scénario**

Même scénario métier (**Panier e-commerce**).

Services impliqués :

- `PanierService`
- `StockService`
- `PaieementService`
- `NotificationService`

Événements impliqués :

`ArticleAjoute`, `StockReserve`, `PaieementEffectue`, `PanierExpire`, `StockLibere`

2. Conception de la saga chorégraphiée**Coordination sans orchestrateur central :**

Chaque service réagit aux événements reçus.

Exemple de flux :

1. `PanierService` publie `ArticleAjoute`
2. `StockService` consomme `ArticleAjoute`, réserve le stock, publie `StockReserve`
3. `PaieementService` consomme `StockReserve`, tente le paiement, publie `PaieementEffectue` ou `PaieementEchoue`
4. En cas d'échec, `StockService` consomme `PaieementEchoue` et publie `StockLibere`

Diagramme de séquence établi pour représenter ce flux.

3. Mise en œuvre technique

Les producteurs / consommateurs existants sont étendus pour :

- Réagir à plus d'événements
- Publier des événements compensatoires (ex : libération de stock)

Gestion des échecs par compensation automatique via événements.

4. Tests et observabilité

Scénarios simulés :

- **Réussite complète** (panier payé et validé)
- **Échec partiel** (paiement refusé, stock libéré)

Métriques Prometheus ajoutées :

- Sagas démarrées
- Sagas réussies
- Sagas échouées

Tableau Grafana montrant :

- Latence entre événements
- Taux de succès / échec
- Volumes d'événements par type